

LOCKSYNC: DISTRIBUTED LOCKING SYSTEM

Submitted By

Aniket Rai
(B.Tech CSE 2024-28)

Problem Statement

In modern distributed systems, multiple applications and services often need access to shared resources such as databases, files, inventory records, payment transactions, and configuration settings. When several services attempt to access the same resource simultaneously, race conditions may occur, leading to data inconsistency, duplicate processing, system crashes, and unexpected behavior.

To address these challenges, distributed locking mechanisms are used to ensure that only one process can access a critical resource at a given time. However, implementing a reliable locking system introduces additional challenges such as lock expiration, deadlock prevention, fault tolerance, scalability, and consistency across distributed nodes.

The objective of LockSync is to design and implement a distributed locking service that provides secure lock acquisition, lock renewal, automatic expiration, and lock release functionality while maintaining consistency and reliability.

Proposed Solution

LockSync is a centralized distributed lock management service built using Python and FastAPI. The system allows clients to acquire locks on shared resources using unique ownership tokens. Each lock contains an expiration period (TTL) that prevents deadlocks by automatically releasing inactive locks.

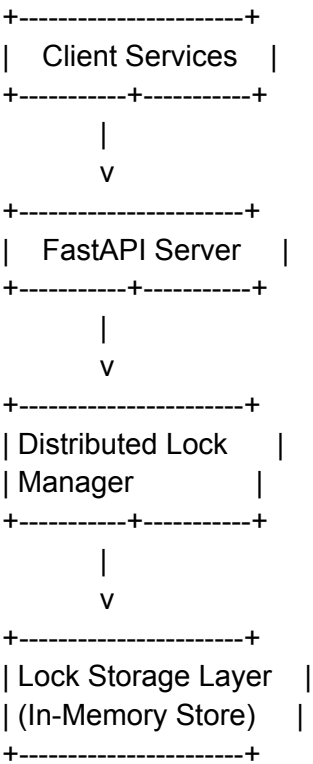
The proposed solution includes:

- Lock Acquisition API
- Lock Renewal API
- Lock Release API
- Lock Monitoring API
- Token-Based Ownership Validation
- Automatic Lock Expiration
- RESTful Service Architecture
- Extensible Design for Future Distributed Deployment

The system ensures that only one client can hold a lock on a resource at any given time while providing a simple and scalable architecture.

System Architecture

High-Level Architecture



Architecture Components

Client Services

Applications that request locks before accessing shared resources.

FastAPI Server

Acts as the API gateway and receives lock-related requests.

Distributed Lock Manager

Handles lock acquisition, renewal, release, ownership verification, and expiration management.

Lock Storage Layer

Stores active lock information including resource identifiers, ownership details, tokens, and expiration timestamps.

Module Description

1. lock_manager.py

This module contains the core business logic of the application.

Responsibilities

- Lock Acquisition
- Lock Renewal
- Lock Release
- Expired Lock Cleanup
- Ownership Verification

Key Functions

- `acquire_lock()`
 - `release_lock()`
 - `renew_lock()`
 - `cleanup_expired_locks()`
 - `show_locks()`
-

2. server.py

This module exposes REST APIs using FastAPI.

Responsibilities

- Request Processing
- API Routing
- Response Handling
- Swagger Documentation Generation

API Endpoints

POST /acquire-lock

POST /renew-lock

POST /release-lock

GET /locks

Database Design

Current Implementation

The current implementation uses an in-memory Python dictionary to store lock information.

Example:

```
{
  "resource_test": {
    "owner": "service_A",
    "token": "72feda50-d398-422e-bb34-101162a428bc",
    "expiry": 1781514512
  }
}
```

Proposed MongoDB Design

Collection: locks

Field	Description
resource_id	Unique resource identifier
owner_id	Lock owner
token	Unique lock token
created_at	Creation timestamp
expires_at	Expiration timestamp
status	Active / Released

Sample Document

```
{
  "_id": "1",
  "resource_id": "resource_101",
  "owner_id": "service_A",
  "token": "abc123xyz",
  "created_at": "2026-06-15",
  "expires_at": "2026-06-15T12:30:00",
  "status": "active"
}
```

Advantages

- High scalability
- Fast read/write performance
- Easy replication support
- Flexible schema design

Technology Stack

Programming Language

Python 3.12

Backend Framework

FastAPI

API Server

Uvicorn

Documentation

Swagger UI

Version Control

Git & GitHub

Data Structures

Python Dictionary

Future Database

MongoDB

Implementation Details

Lock Acquisition Process

1. Client sends resource ID and owner ID.
2. System checks whether resource is already locked.
3. If available, a UUID token is generated.
4. Lock metadata is stored.
5. Token is returned to the client.

Lock Renewal Process

1. Client submits resource ID and token.
2. Ownership verification is performed.
3. Expiration time is extended.
4. Success response is returned.

Lock Release Process

1. Client submits resource ID and token.
2. Token validation occurs.
3. Lock is removed.
4. Resource becomes available.

Lock Expiration Process

1. System periodically checks expiration timestamps.
2. Expired locks are identified.
3. Expired locks are automatically removed.
4. Resources become available for new requests.

Security Mechanism

Lock ownership is protected using UUID-based tokens.

Only the lock owner possessing the valid token can:

- Renew a lock
 - Release a lock
-

Screenshots

Figure 1: Swagger UI Dashboard

LockSync Distributed Locking Service 0.1.0 OAS 3.1

/openapi.json

default		^
GET	/ Home	▼
POST	/acquire-lock Acquire Lock	▼
POST	/release-lock Release Lock	▼
POST	/renew-lock Renew Lock	🔗 ▼
GET	/locks Get Locks	▼
Schemas		^
HTTPValidationError > Expand all object		
ValidationError > Expand all object		

Description:

The Swagger interface provides interactive API testing and documentation.

Figure 2: Lock Acquisition API

Parameters

Cancel

Name	Description
resource_id * required string (query)	resource_test
owner_id * required string (query)	SERVICE_A
ttl integer (query)	300

Execute

Clear

Responses

Curl

```
curl -X 'POST' \
  'http://127.0.0.1:8080/acquire-lock?resource_id=resource_test&owner_id=SERVICE_A&ttl=300' \
  -H 'accept: application/json' \
  -d ''
```

Request URL

```
http://127.0.0.1:8080/acquire-lock?resource_id=resource_test&owner_id=SERVICE_A&ttl=300
```

Server response

Code	Details
200	Response body <pre>{ "status": "success", "token": "c05f997b-1de4-4d08-80b1-33fdaff45bb1" }</pre> Download Response headers <pre>content-length: 67 content-type: application/json date: Mon, 15 Jun 2026 09:18:47 GMT server: uvicorn</pre>

Responses

Code	Description	Links
------	-------------	-------

Description:
Demonstrates successful lock acquisition and token generation.

Figure 3: View Active Locks

GET /locks Get Locks

Parameters

No parameters

Execute Clear

Responses

Curl

```
curl -X 'GET' \
  'http://127.0.0.1:8000/locks' \
  -H 'accept: application/json'
```

Request URL

http://127.0.0.1:8000/locks

Server response

Code	Details
200	Response body <pre>{ "resource_test": { "owner": "SERVICE_A", "token": "c05f997b-1de4-4d08-80b1-33fdaff45bb1", "expiry": 1701515427.725035 } }</pre> Response headers <pre>content-length: 113 content-type: application/json date: Mon, 15 Jun 2026 09:19:19 GMT server: uvicorn</pre>

Responses

Code	Description	Links
200	Successful Response	No links

Media type: application/json

Controls Accept header.

Example Value Schema

Description:
Displays active lock information and ownership details.

Figure 4: Lock Renewal API

POST/renew-lockRenew Lock

Parameters

Cancel

Name	Description
resource_id * required string (query)	resource_test
token * required string (query)	c05f997b-1de4-4d08-80b1-33fdaff45bb1
ttl integer (query)	30

Execute

Clear

Responses

Curl

```
curl -X 'POST' \  
  'http://127.0.0.1:8000/renew-lock?resource_id=resource_test&token=c05f997b-1de4-4d08-80b1-33fdaff45bb1&ttl=30' \  
  -H 'accept: application/json' \  
  -d ''
```

Request URL

```
http://127.0.0.1:8000/renew-lock?resource_id=resource_test&token=c05f997b-1de4-4d08-80b1-33fdaff45bb1&ttl=30
```

Server response

Code	Details
200	Response body <pre>{ "renewed": true }</pre> Download Response headers <pre>content-length: 16 content-type: application/json date: Mon, 15 Jun 2026 09:28:34 GMT server: uvicorn</pre>

Responses

Code	Description	Links
------	-------------	-------

Description:
Demonstrates successful lock renewal and TTL extension.

Figure 5: Lock Release API

POST

/release-lock

Release Lock

⌵

Parameters

Cancel

Name	Description
resource_id * required	
string	resource_test
(query)	
token * required	
string	c05f997b-1de4-4d08-80b1-33fdaff45bb1
(query)	

Execute

Clear

Responses

Curl

```
curl -X 'POST' \
'http://127.0.0.1:8080/release-lock?resource_id=resource_test&token=c05f997b-1de4-4d08-80b1-33fdaff45bb1' \
-H 'accept: application/json' \
-d ''
```

Request URL

```
http://127.0.0.1:8080/release-lock?resource_id=resource_test&token=c05f997b-1de4-4d08-80b1-33fdaff45bb1
```

Server response

Code	Details
200	Response body <pre>{ "released": true }</pre> Download Response headers <pre>content-length: 17 content-type: application/json date: Mon, 15 Jun 2026 09:21:23 GMT server: uvicorn</pre>

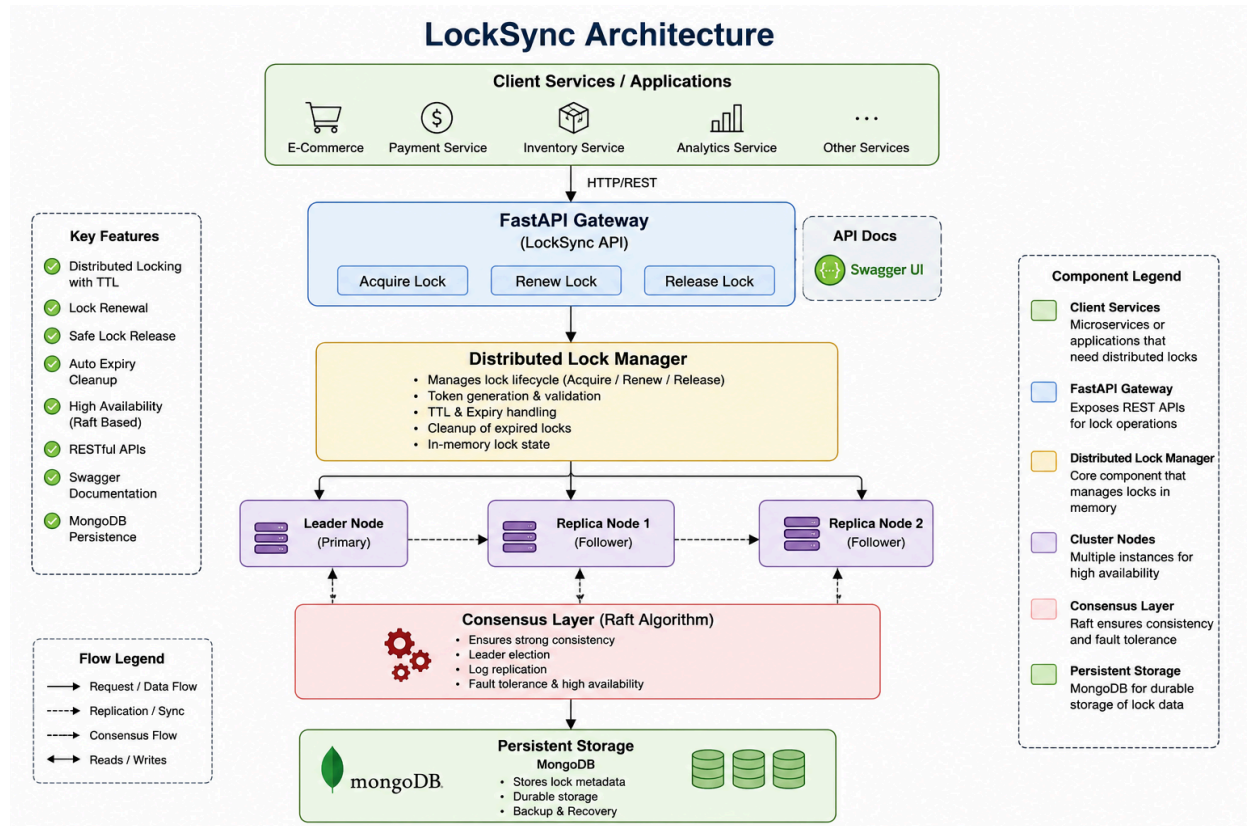
Responses

Code	Description	Links
200	Successful Response	No links

Description:

Shows successful lock release and resource unlock operation.

Figure 6: Architecture Diagram



Description:

Illustrates the overall LockSync system architecture and component interactions.

Future Scope

The current implementation serves as a foundational distributed locking service. Future enhancements can transform it into a production-ready distributed coordination platform.

MongoDB Persistence

Store lock information permanently and support service restarts.

Redis Integration

Implement Redis-based distributed locking for higher performance.

Raft Consensus Algorithm

Enable leader election and distributed consensus.

Replication Layer

Synchronize lock information across multiple nodes.

Kubernetes Deployment

Deploy LockSync in cloud-native environments.

Multi-Region Support

Support globally distributed lock management.

Monitoring Dashboard

Create a web interface for lock monitoring and analytics.

AI-Assisted Coordination

Implement intelligent lock scheduling and optimization mechanisms.

Conclusion

LockSync successfully demonstrates the design and implementation of a distributed locking system capable of managing access to shared resources in distributed environments. The project provides lock acquisition, renewal, release, and automatic expiration mechanisms through a RESTful API architecture.

The implementation highlights key distributed systems concepts such as mutual exclusion, ownership validation, timeout handling, and fault tolerance. While the current version uses in-memory storage, the architecture is extensible and can be enhanced with MongoDB, Redis, consensus algorithms, and distributed replication mechanisms to support enterprise-scale deployments.

The project provides valuable practical experience in distributed systems design, API development, concurrency management, and scalable software architecture.